

NSU ClubHub: A Centralized Club Management & Engagement Portal for North South University

M.A. Rahman Junaid¹, Shabab Alam Pean², Mohammad Safayet Alif³, Syed Mohim Rahman Preo⁴
Dept. of Electrical and Computer Engineering

North South University

Dhaka, Bangladesh

IDs: 2311947642¹, 2231829042², 2232562642³, 2232812642⁴

Dr. Mohammad Shifat-E-Rabbi

Assistant Professor

Dept. of Electrical and Computer Engineering

North South University

Dhaka, Bangladesh

Abstract—NSU ClubHub is a full-stack web platform designed to centralize and improve student club management at North South University, where operations are currently handled through fragmented tools like social media, emails, and spreadsheets. Despite having over thirty active clubs, this lack of a unified system makes coordination, event management, and recruitment inefficient. The platform offers a single system for club discovery, event management, member recruitment, and administrative control. It is built with Next.js 15 for the frontend, Express.js with TypeScript for the backend, PostgreSQL for data storage, and Prisma as the ORM. Key features include role-based access control (Student, Club Admin, Super Admin), JWT authentication, QR-based event ticketing, structured recruitment workflows, real-time notifications, and an AI assistant. Evaluation through functional testing, OWASP-based security checks, performance analysis, and usability testing (SUS score: 81.4) indicates that the system is secure, efficient, and suitable for institutional deployment.

Index Terms—club management system, university portal, Next.js, Express.js, PostgreSQL, RBAC, JWT authentication, QR ticketing, web application, student engagement

I. INTRODUCTION

Student clubs and co-curricular organizations are vital for university life, fostering leadership, networking, academic growth, and engagement. At North South University (NSU) in Dhaka, over thirty clubs operate in areas such as robotics, debate, entrepreneurship, photography, and cultural activities, but their management is largely informal and fragmented. Currently, clubs rely on disconnected tools like Facebook pages, Google Forms, spreadsheets, paper records, and messaging apps, which creates inefficiencies in coordination, recruitment, event management, and administrative oversight while also reducing visibility for university authorities and lacking standardized accountability. NSU ClubHub addresses these challenges by providing a centralized platform for club registration, discovery, event management, ticketing, recruitment, notifications, and administration. Designed specifically for NSU, it implements role-based access control to reflect club hierarchies and ensure proper governance. The system

emphasizes scalability, security, and usability, making it suitable for a growing student population. This paper presents the design, implementation, and evaluation of NSU ClubHub as a unified solution for improving club management in higher education

II. RELATED WORK

A. Commercial Club Management Platforms

A number of commercial solutions exist that attempt to address the broader problem of student organization management at the university level. CampusGroups [2] is among the most widely deployed, offering features such as event scheduling, member directories, financial tracking, and analytics dashboards. While feature-rich, CampusGroups is a Software-as-a-Service (SaaS) product with licensing costs that render it impractical for institutions in developing economies such as Bangladesh. Moreover, its configurability is constrained by its generic design, which does not accommodate institution-specific workflows, nomenclature, or organizational hierarchies. OrgSync, now merged into Campus Labs (Anthology), similarly provides a comprehensive suite of co-curricular management tools but shares the same limitations of cost and generic design [3].

B. Learning Management Systems as Partial Solutions

Learning Management Systems (LMS) such as Moodle [4] and Blackboard have occasionally been repurposed to manage student organizations, leveraging their existing course-group infrastructure. While these platforms offer authentication integration and document sharing, they are fundamentally designed for pedagogical workflows and lack primitives necessary for club management, such as event ticketing, open recruitment pipelines, or multi-club discovery mechanisms. Adapting an LMS to serve as a club portal typically requires extensive plugin development and results in a user experience that remains pedagogically framed, which is inappropriate for co-curricular contexts.

C. Custom Institutional Portals in Literature

A growing body of academic literature documents custom-built institutional portals designed to address specific university management challenges. Kaur and Singh [5] developed a campus activity management system for an Indian university using a PHP-MySQL stack, demonstrating the value of purpose-built solutions but relying on a technology stack that lacks the scalability and developer ergonomics of modern frameworks. Rahman et al. [6] proposed a mobile-first student activity tracker for Bangladeshi universities, emphasizing the importance of low-bandwidth optimizations; however, their work did not address administrative role hierarchies or event ticketing. More recently, web applications built on the MERN stack (MongoDB, Express, React, Node.js) have been documented for club and event management [7], but these works typically lack formal security evaluations and do not address the multi-tenancy requirements of large university environments.

D. Authentication and Access Control in Educational Portals

The implementation of secure authentication and role-based access control in educational web applications has received dedicated attention in the literature. JSON Web Tokens (JWT) have emerged as the dominant stateless authentication mechanism for single-page applications due to their compactness, verifiability, and compatibility with distributed architectures [8]. RBAC models, as formalized by Ferraiolo et al. [9], provide a principled framework for mapping organizational roles to system permissions, and their application to university portals has been explored by several researchers. ClubHub's implementation draws upon these established principles while extending them to accommodate the specific three-tier hierarchy of NSU's club governance structure.

E. Positioning of NSU ClubHub

In contrast to the works surveyed above, NSU ClubHub occupies a distinctive position: it is an open-source, institution-specific, full-stack web application built on a contemporary technology stack (Next.js 15, Express.js, TypeScript, PostgreSQL, Prisma), incorporating features—specifically QR-based ticketing, a structured recruitment pipeline, and an integrated AI assistant—that have not been simultaneously addressed in prior academic works on university club management systems. Table I presents a structured feature comparison between ClubHub and representative existing solutions.

III. SYSTEM ARCHITECTURE

A. Architectural Overview

NSU ClubHub adopts a layered, service-oriented architecture organized into three principal tiers: the presentation tier, the application logic tier, and the data persistence tier. This separation of concerns ensures that each layer can be developed, tested, and scaled independently. The frontend and backend are decoupled, communicating exclusively through a well-defined RESTful API, which enables future substitution

of either layer without disrupting the other. The overall architecture is depicted in Fig. 1.

Fig. 1. System Architecture of NSU ClubHub. The diagram illustrates the three-tier structure comprising the Next.js 15 frontend, the Express.js/TypeScript REST API backend, and the PostgreSQL database accessed via Prisma ORM, along with cross-cutting concerns including JWT authentication, RBAC middleware, and the AI assistant module.

B. Frontend Tier

The presentation tier is implemented using Next.js 15, a React-based framework that supports both server-side rendering (SSR) and static site generation (SSG). The adoption of Next.js was driven by several considerations: its App Router paradigm, introduced in Next.js 13 and matured in version 15, enables fine-grained control over rendering strategies on a per-route basis; its built-in image optimization reduces bandwidth consumption; and its first-class TypeScript support aligns with the project's type-safety objectives. The frontend communicates with the backend exclusively via HTTP requests to the REST API, with authentication tokens stored in HTTP-only cookies to mitigate cross-site scripting (XSS) risks.

C. Backend Tier

The application logic tier is implemented as an Express.js server written in TypeScript. Express.js was selected for its minimalism, extensive middleware ecosystem, and the team's familiarity with the Node.js runtime. TypeScript adds compile-time type checking, which substantially reduces the incidence of runtime errors in a codebase where data shapes are complex and relationships between entities are numerous. The backend is organized following the Model-View-Controller (MVC) pattern adapted for API-first design: routes define URL structures, controllers encapsulate business logic, and services abstract database operations. Middleware functions handle cross-cutting concerns including authentication verification, role authorization, request validation, error normalization, and request logging.

D. Data Persistence Tier

PostgreSQL serves as the primary relational database, selected for its ACID compliance, robust support for complex queries, and maturity as a production-grade database system. Prisma ORM mediates all interactions between the application tier and the database, providing a type-safe query builder that generates TypeScript types from the database schema. This integration eliminates a category of runtime errors common in dynamically typed ORMs and enables autocompletion in the development environment. Database migrations are managed through Prisma Migrate, ensuring schema changes are versioned and reproducible.

E. Cross-Cutting Concerns

Several architectural concerns span multiple tiers. Authentication is implemented using JSON Web Tokens: upon successful login, the server issues a signed JWT containing the user's

TABLE I
FEATURE COMPARISON OF CLUB/ACTIVITY MANAGEMENT SYSTEMS

Feature	CampusGroups	Moodle (adapted)	Prior NSU Projects	NSU ClubHub
Club Discovery Portal	✓	Partial	✓	✓
Role-Based Access Control (3-tier)	✓	Partial	Partial	✓
JWT Stateless Authentication	✓	×	×	✓
QR-Code Event Ticketing	✓	×	×	✓
Structured Recruitment Pipeline	✓	×	×	✓
Real-Time Notifications	✓	Partial	×	✓ (polling)
Integrated AI Assistant	Partial	×	×	✓
Open Source / Self-Hosted	×	✓	✓	✓
Institution-Specific Customization	×	Partial	✓	✓
RESTful API with TypeScript	×	×	×	✓
PostgreSQL + Prisma ORM	×	×	×	✓
SUS Usability Score (≥ 80)	Unknown	Unknown	Unknown	✓ (81.4)
Zero Licensing Cost	×	✓	✓	✓

identifier, role, and token expiry; this token is validated on every subsequent request by an authentication middleware. The AI assistant module integrates with an external large language model API, processing natural language queries from users and returning contextually relevant responses about club activities, event schedules, and platform navigation. Notification delivery currently employs a polling-based mechanism, where the frontend periodically queries a dedicated notifications endpoint; this approach was chosen for implementation simplicity and will be upgraded to WebSocket-based push delivery in a future iteration.

IV. FRONTEND DESIGN

A. Technology Selection and Rationale

The frontend of NSU ClubHub is built with Next.js 15 utilizing the App Router, which represents a significant evolution from the Pages Router paradigm. The App Router introduces React Server Components (RSC), enabling components to be rendered on the server without shipping their JavaScript to the client, thereby reducing the total JavaScript bundle size and improving Time to First Contentful Paint (FCP). For NSU’s student population, a significant proportion of whom access the internet via mobile devices on constrained bandwidth connections, this performance characteristic is especially valuable. Tailwind CSS is employed for styling, providing a utility-first approach that accelerates UI development and enforces visual consistency through design tokens defined in the project’s configuration file.

B. Component Architecture and State Management

The frontend component hierarchy is organized into three categories: page components, feature components, and primitive components. Page components correspond to Next.js routes and orchestrate the composition of feature components. Feature components encapsulate self-contained UI domains such as the event card carousel, the recruitment application form, or the notification drawer. Primitive components, such

as buttons, input fields, and modal dialogs, are styled with Tailwind and adhere to accessibility guidelines (WCAG 2.1 AA). Global client-side state is managed using Zustand, a lightweight state management library selected over Redux for its minimal boilerplate and intuitive API. Server state—data fetched from the API—is managed using TanStack Query (React Query), which provides caching, background refetching, and optimistic updates.

C. Routing and Role-Conditioned Navigation

Next.js’s App Router enables route groups and nested layouts, which ClubHub exploits to implement role-conditioned navigation. Authenticated routes are protected by a middleware function that inspects the JWT stored in the HTTP-only cookie; unauthenticated requests are redirected to the login page. Once authenticated, the navigation menu rendered in the root layout adapts its items based on the user’s role: students see a discovery-oriented interface; club administrators see management tools for their specific club; super administrators see a system-wide oversight dashboard. This conditional rendering is implemented declaratively through a custom `RoleGuard` component that wraps sensitive route segments.

D. Responsive Design and Accessibility

ClubHub is designed as a mobile-first application, with breakpoint-specific layouts defined through Tailwind’s responsive utility variants. All interactive elements are keyboard-navigable, and ARIA attributes are applied to dynamic components such as modals and dropdown menus to ensure compatibility with screen readers. Color contrast ratios across the application meet WCAG 2.1 AA standards, verified using the browser’s built-in accessibility audit tool during the testing phase.

V. BACKEND DESIGN

A. Server Framework and Middleware Stack

The backend API server is implemented using Express.js version 4.x with TypeScript. The middleware stack is carefully ordered to ensure that cross-cutting concerns are applied in the correct sequence. Incoming requests pass first through the CORS middleware, which enforces origin restrictions permitting only the ClubHub frontend domain. They then traverse the JSON body parser, the request logger (using Morgan), the rate limiter (using express-rate-limit, configured to 100 requests per IP per 15-minute window), the authentication middleware (JWT verification), the RBAC authorization middleware, and finally reach the route handler. Error propagation is managed through a centralized error-handling middleware registered at the bottom of the middleware stack, which normalizes all thrown errors into a consistent JSON response format with appropriate HTTP status codes.

B. Authentication Architecture

JWT-based authentication is implemented in two phases. During registration, the user's password is hashed using bcrypt with a salt factor of twelve before storage; this ensures that even a database breach does not expose plaintext credentials. During login, the submitted password is compared against the stored hash using bcrypt's constant-time comparison function; upon successful verification, the server signs a JWT using a secret key stored in an environment variable, embedding the user's ID, email, and role as claims with a 24-hour expiry. Refresh token logic is implemented using a separate, longer-lived token stored in the database, enabling session renewal without requiring re-authentication. Logout is handled by invalidating the refresh token server-side and clearing the client-side cookie.

C. Role-Based Access Control Implementation

The RBAC system defines three roles with hierarchically scoped permissions: Student, Club Admin, and Super Admin. The authorization middleware extracts the role claim from the validated JWT and compares it against the role requirements annotated on each route. Club Admin permissions are further scoped by club membership: a club admin can only manage events, recruitment pipelines, and members belonging to their specific club, enforced through a club ownership check in the relevant service functions. Super Admins have unrestricted access to all system resources, including the ability to approve or reject club registration requests, suspend users, and view aggregate analytics across all clubs.

D. Service Layer and Business Logic

The service layer abstracts all database access behind a set of typed service functions organized by domain entity (UserService, ClubService, EventService, RecruitmentService, NotificationService, TicketService). Each service function accepts typed input parameters, performs the necessary database operations through Prisma, applies business rules (such as

preventing duplicate event registrations or enforcing recruitment application deadlines), and returns typed output. This encapsulation ensures that business logic is not scattered across route handlers and can be independently unit tested.

VI. DATABASE DESIGN

A. Schema Overview

The PostgreSQL database schema is defined and managed through Prisma's schema definition language (SDL). The schema comprises twelve principal models: User, Club, ClubMembership, Event, EventRegistration, Ticket, RecruitmentCycle, RecruitmentApplication, Notification, NotificationRecipient, AuditLog, and AIConversation. These models are interconnected through a combination of one-to-one, one-to-many, and many-to-many relationships, all enforced at the database level through foreign key constraints.

B. Core Entity Relationships

The User model stores authentication credentials (hashed password, email), profile information (name, student ID, department, profile picture URL), and the global role enumeration (STUDENT, CLUB_ADMIN, SUPER_ADMIN). A user's membership in a specific club is represented by the ClubMembership join model, which includes the user's role within that club (member, executive, president) and the timestamp of joining. This design correctly models the many-to-many relationship between users and clubs while capturing role-specific membership metadata.

The Event model captures event metadata including title, description, venue, start and end timestamps, capacity, banner image URL, and the organizing club's foreign key. Event registrations are tracked through the EventRegistration model, which links users to events and records registration timestamp and payment status where applicable. Each confirmed event registration generates a corresponding Ticket record, which stores a unique QR code payload (a UUID embedded in a signed string), validation status, and the timestamp of QR scan.

C. Recruitment Pipeline Schema

The recruitment subsystem is modeled through two entities: RecruitmentCycle and RecruitmentApplication. A RecruitmentCycle is associated with a specific club and defines the recruitment period, available roles, and the pipeline stages (e.g., Application Submitted, Shortlisted, Interview Scheduled, Selected, Rejected). A RecruitmentApplication links a student user to a recruitment cycle and tracks the current pipeline stage as a state machine, with stage transitions recorded in the AuditLog for traceability.

D. Notification and Audit Models

The Notification model represents a notification event (e.g., event approval, recruitment status update) with a type enumeration, a payload JSON field for dynamic content, and the generating entity’s identifier. The NotificationRecipient join model associates notifications with target users and records read status and timestamp. The AuditLog model records all state-modifying operations performed by Club Admins and Super Admins, capturing the actor, the action type, the target entity, and a JSON snapshot of the before and after states, supporting compliance and accountability requirements.

VII. FEATURE MODULES AND API DESIGN

A. Club Discovery and Management Module

The Club Discovery module provides students with a searchable, filterable directory of all NSU clubs. Each club entry displays the club’s name, category, banner image, brief description, current membership count, and a call-to-action button for joining or following. The discovery interface supports full-text search across club names and descriptions, implemented using PostgreSQL’s `to_tsvector` and `to_tsquery` functions for efficient server-side text search. Filtering is supported by club category (academic, cultural, technical, social, sports) and membership status. Club administrators can manage their club’s profile, including updating the banner, description, executive listing, and social links, through a dedicated management dashboard.

B. Event Management and QR Ticketing Module

The Event Management module allows club administrators to create, publish, and manage events with full metadata. Upon event creation, the event enters a pending state requiring Super Admin approval before becoming visible to students—this governance layer prevents unauthorized or inappropriate events from being publicized. Once approved, students can register for events up to the defined capacity limit. Upon successful registration, the backend generates a unique ticket record with a cryptographically signed QR code payload using the `uuid` and `jsonwebtoken` libraries. The QR code is rendered on the frontend using the `qrcode.react` library and delivered to the student via the platform’s notification system.

At the event venue, club administrators use the ClubHub mobile-responsive interface to scan attendees’ QR codes using the device camera via the `jsQR` library integrated into the frontend. The scanned payload is transmitted to the backend’s ticket validation endpoint, which verifies the cryptographic signature, checks that the ticket has not been previously scanned, and marks it as validated. This system eliminates duplicate entry exploitation and provides real-time attendance tracking accessible to event organizers.

C. Recruitment Pipeline Module

The Recruitment Pipeline module provides a structured, multi-stage workflow for club recruitment cycles. Club administrators initiate a new recruitment cycle by specifying the cycle name, application window (open and close dates), available roles, and the sequence of pipeline stages. Students browse open recruitment cycles on the discovery page and submit applications through an in-platform form that captures motivational statements and optionally attaches a portfolio URL. Applications transition through pipeline stages as club administrators review them; each stage transition triggers a notification to the applicant. The pipeline interface for club administrators presents a Kanban-style board with drag-and-drop functionality, implemented using the `@dnd-kit/core` library, enabling intuitive management of large applicant pools.

D. AI Assistant Module

The AI assistant is accessible via a chat interface embedded in the platform’s navigation sidebar. The assistant is powered by an external large language model API and is provided with a system prompt that contextualizes it as a knowledgeable guide for NSU ClubHub, capable of answering questions about platform features, club activities, event schedules, and general navigation. The current implementation is not retrieval-augmented; the assistant operates from its pre-trained knowledge supplemented by the system prompt. Future iterations will integrate retrieval-augmented generation (RAG) to ground the assistant’s responses in live database content.

E. REST API Design and Endpoints

The ClubHub backend exposes a RESTful API organized under the `/api/v1` base path. Table II documents the principal endpoints across all feature modules, including the HTTP method, route pattern, required role, and functional description.

VIII. EVALUATION AND RESULTS

A. Evaluation Methodology

The evaluation of NSU ClubHub was conducted across four complementary dimensions: functional correctness testing, security vulnerability assessment aligned with the OWASP Top 10 framework, quantitative performance benchmarking, and a structured usability study using the System Usability Scale (SUS) instrument. This multi-faceted evaluation strategy is consistent with best practices in software engineering research [11] and provides a holistic assessment of the platform’s fitness for production deployment. All testing was conducted in a staging environment closely mirroring the intended production configuration, with the backend deployed on a virtual private server running Ubuntu 22.04 LTS and the frontend hosted on Vercel’s edge network.

TABLE II
NSU CLUBHUB REST API ENDPOINT REFERENCE

Method	Route	Required Role	Description
POST	/api/v1/auth/register	Public	Register a new student account with email and password.
POST	/api/v1/auth/login	Public	Authenticate a user; returns signed JWT and refresh token.
POST	/api/v1/auth/refresh	Public	Issue a new access token using a valid refresh token.
GET	/api/v1/clubs	Authenticated	Retrieve paginated list of all clubs with search and filter support.
POST	/api/v1/clubs	Super Admin	Create and register a new club in the system.
GET	/api/v1/clubs/:id	Authenticated	Retrieve full details of a specific club by ID.
PATCH	/api/v1/clubs/:id	Club Admin	Update club profile fields (description, banner, social links).
GET	/api/v1/events	Authenticated	Retrieve all approved, upcoming events with optional filters.
POST	/api/v1/events	Club Admin	Create a new event (enters pending approval state).
PATCH	/api/v1/events/:id/approve	Super Admin	Approve a pending event, making it publicly visible.
POST	/api/v1/events/:id/register	Student	Register the authenticated student for an event.
GET	/api/v1/tickets/:id/validate	Club Admin	Validate a QR ticket payload; returns validation status.
GET	/api/v1/recruitment	Authenticated	List all open recruitment cycles across all clubs.
POST	/api/v1/recruitment	Club Admin	Create a new recruitment cycle for the admin's club.
POST	/api/v1/recruitment/:id/apply	Student	Submit an application to a specific recruitment cycle.
PATCH	/api/v1/recruitment/applications/:id	Club Admin	Advance or reject an application in the pipeline.
GET	/api/v1/notifications	Authenticated	Retrieve unread notifications for the authenticated user.
POST	/api/v1/ai/chat	Authenticated	Submit a message to the AI assistant; returns LLM response.
GET	/api/v1/admin/analytics	Super Admin	Retrieve aggregate platform analytics (users, events, clubs).

B. Functional Testing

Functional testing was conducted at three levels: unit testing, integration testing, and end-to-end (E2E) testing. Unit tests were written using Jest and covered all service layer functions, with particular attention to boundary conditions in business logic (e.g., event registration at full capacity, recruitment application submission after deadline, QR ticket re-scan attempts). A total of 147 unit test cases were written, achieving a code coverage of 84.3% across the service layer. Integration tests verified the correct interaction between route handlers, middleware functions, and service functions using Supertest, which enables HTTP assertions against the live Express application. Forty-three integration tests were authored, exercising each REST API endpoint with both valid and invalid inputs, verifying response status codes, response body schemas, and database state changes.

End-to-end testing was performed using Playwright, which drives a real browser against the fully integrated application stack. E2E test scenarios covered the seven primary user journeys: student registration and login, club discovery and profile viewing, event discovery and registration with QR ticket generation, QR ticket scanning and validation, recruitment cycle browsing and application submission, club administrator event creation and management, and super administrator club approval workflow. All 31 E2E test cases passed successfully upon the final evaluation build, confirming end-to-end correct-

ness of the critical user paths.

C. Security Testing

Security evaluation followed the OWASP Top 10 (2021) vulnerability taxonomy [10], systematically assessing ClubHub against the ten most critical web application security risks. The assessment methodology combined automated scanning using OWASP ZAP (Zed Attack Proxy) in active scan mode with manual penetration testing exercises targeting the authentication and authorization subsystems.

With respect to **Injection (A03:2021)**, all database queries are executed exclusively through Prisma's parameterized query interface, which prevents SQL injection by construction. Input validation middleware using the Zod schema validation library ensures that all incoming request payloads conform to expected types and ranges before reaching the service layer, mitigating injection risks in derived query parameters. The OWASP ZAP active scan found zero high-severity injection vulnerabilities.

With respect to **Broken Authentication (A07:2021)**, the implementation of bcrypt password hashing with a work factor of twelve, constant-time comparison, HTTP-only JWT cookies, and refresh token rotation collectively address the primary authentication risks. Rate limiting on the login endpoint (five attempts per minute per IP before temporary lockout) mitigates brute-force attacks. Manual penetration testing confirmed that JWT tokens cannot be forged without knowledge of the

server's secret key, and that expired tokens are correctly rejected.

With respect to **Broken Access Control (A01:2021)**, the RBAC middleware was subjected to systematic privilege escalation testing. Attempts to access Club Admin endpoints using Student-role JWTs were uniformly rejected with 403 Forbidden responses. Attempts to access another club's management endpoints using a Club Admin JWT scoped to a different club were likewise rejected at the service layer. No access control bypass vulnerabilities were identified.

With respect to **Security Misconfiguration (A05:2021)**, the application's CORS policy is strictly scoped to the production frontend domain. HTTP security headers (X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, Content-Security-Policy) are applied via the Helmet.js middleware. Directory listing is disabled on the server, and verbose error messages are suppressed in production mode to prevent information leakage. One medium-severity finding—an overly permissive Content-Security-Policy directive for the AI assistant's external API domain—was identified and subsequently tightened.

With respect to **Cross-Site Scripting (A03 in prior editions)**, all user-generated content displayed in the frontend is escaped by React's default rendering behavior. Instances where HTML content must be rendered (e.g., event descriptions supporting rich text) employ the `DOMPurify` library for sanitization before insertion. The OWASP ZAP passive scan found no reflected or stored XSS vulnerabilities in the production build.

Overall, the security assessment identified zero critical vulnerabilities, one medium vulnerability (subsequently remediated), and three low-severity informational findings, demonstrating a satisfactory security posture for an institutional deployment context.

D. Performance Benchmarking

Performance benchmarking was conducted using Apache JMeter configured to simulate concurrent user loads representing realistic usage patterns. Three load scenarios were defined and executed: a baseline scenario with 50 concurrent users, a moderate scenario with 200 concurrent users, and a stress scenario with 500 concurrent users, each sustaining load for a five-minute duration with a ramp-up period of 30 seconds.

Under the baseline scenario (50 concurrent users), the API exhibited a mean response time of 87 milliseconds with a 95th percentile response time of 134 milliseconds and a throughput of 412 requests per second. Error rate was 0.0%. Under the moderate scenario (200 concurrent users), mean response time increased to 213 milliseconds with a 95th percentile of 389 milliseconds and a throughput of 887 requests per second. Error rate remained at 0.0%, confirming that the application correctly handled moderate production-scale load without service degradation. Under the stress scenario (500 concurrent users), mean response time rose to 621 milliseconds with a 95th percentile of 1,240 milliseconds. Throughput peaked at 1,103 requests per second before the database connection pool

reached saturation. An error rate of 2.1% was observed at peak stress, attributable to connection pool exhaustion rather than application logic failures.

Frontend performance was evaluated using Google Lighthouse, yielding a Performance score of 89/100, an Accessibility score of 94/100, a Best Practices score of 96/100, and an SEO score of 91/100. The Time to First Contentful Paint (FCP) was measured at 1.2 seconds, and the Largest Contentful Paint (LCP) at 2.3 seconds on a simulated 4G mobile connection, both within Google's recommended thresholds.

E. Usability Study

A structured usability study was conducted with 28 participants from NSU's student body, including 20 students and 8 club administrators. Participants were representative of the target users in terms of academic year, technical literacy (measured via a pre-study questionnaire), and club involvement. The study was carried out in a controlled lab environment, with each session lasting around 45 minutes. Each participant completed eight predefined tasks: account registration and login, club discovery and follow, event registration with QR ticket download, recruitment application submission (student role), event creation (Club Admin), recruitment pipeline progression, QR ticket scanning for attendance, and AI assistant usage for retrieving club-related information. Task performance was measured through completion rates, errors, and time-on-task. Overall task completion averaged 94.2

IX. DISCUSSION

The evaluation results collectively demonstrate the technical viability and user acceptance of NSU ClubHub as an institutional club management platform. Functional testing confirms system reliability, with 84.3

X. LIMITATIONS

Despite its strengths, NSU ClubHub has several limitations. The notification system uses client-side polling every 30 seconds, which causes delayed updates and unnecessary server load, and would be better replaced with WebSockets or Server-Sent Events for real-time communication. The AI assistant also lacks retrieval-augmented generation, so it cannot access live data such as current events or recruitment status, limiting its usefulness for dynamic queries. In addition, the platform does not include financial management features like fee collection, budgeting, or ticket monetization, which were excluded to keep the scope focused. The QR-based ticket system depends on browser camera access, which can create usability issues due to permission restrictions on some devices. Finally, the system supports only English, and adding Bengali language support would improve accessibility for a broader range of users.

XI. FUTURE WORK

Several well-defined directions exist for extending and improving NSU ClubHub beyond its current state. Each represents a meaningful enhancement to either the system's technical capability or its practical utility for stakeholders.

WebSocket-Based Real-Time Notifications: The most immediately impactful technical improvement would be the replacement of the polling-based notification mechanism with a WebSocket or Server-Sent Events (SSE) architecture. Socket.IO, which abstracts over WebSocket with fallback support, is the candidate library for this integration. Real-time notifications would eliminate the 30-second delivery latency and reduce server load by removing unnecessary polling requests from idle clients. This enhancement is particularly important for time-sensitive communications such as event capacity warnings, last-call recruitment notifications, and QR validation alerts.

Retrieval-Augmented Generation for the AI Assistant: The AI assistant’s practical utility would be substantially expanded through the implementation of a Retrieval-Augmented Generation (RAG) pipeline. In this architecture, live database content—club profiles, event listings, recruitment cycles, member directories—would be indexed into a vector store (such as pgvector, a PostgreSQL extension for vector similarity search). At query time, the user’s message would be embedded and used to retrieve the most semantically relevant documents from the vector store, which would then be injected into the LLM’s context window as grounding material. This would enable the assistant to accurately answer questions such as “Which clubs are recruiting this month?” or “What events are happening next week?”—queries that are currently beyond its capability.

Analytics Dashboard Enhancement: The current analytics available to super administrators are limited to aggregate counts of users, clubs, and events. A richer analytics layer would include time-series visualizations of platform engagement, cohort analysis of student participation across semesters, event attendance funnel analysis (registered versus attended), and recruitment conversion rate metrics per club and per cycle. These insights would enable university administration to make data-driven decisions about club support and resource allocation.

Inter-University Federation: A longer-term research direction involves extending ClubHub’s architecture to support inter-university club federation, enabling students at partner institutions to discover and participate in joint events. This would require the implementation of an OAuth 2.0 identity federation layer supporting external identity providers from partner universities, a cross-institutional event visibility model with configurable privacy scopes, and API versioning to support clients from heterogeneous institutional deployments.

XII. CONCLUSION

This paper presents NSU ClubHub, a full-stack web application designed to centralize club management and student engagement at North South University. It addresses the fragmentation of club activities across informal channels by providing a unified platform for club discovery, event management, QR-based ticketing, structured recruitment, real-time notifications, and an AI assistant. The system is built using Next.js 15, Express.js with TypeScript, PostgreSQL, and Prisma ORM,

applying modern software engineering practices such as type safety, separation of concerns, role-based access control, and responsive design. Evaluation results show strong performance and reliability, including 84.3

REFERENCES

- [1] A. W. Astin, *What Matters in College? Four Critical Years Revisited*. San Francisco, CA, USA: Jossey-Bass, 1993.
- [2] CampusGroups, “CampusGroups Student Engagement Platform,” 2023. [Online]. Available: <https://www.campusgroups.com>
- [3] Anthology Inc., “Campus Labs by Anthology: Co-Curricular Management Solutions,” 2022. [Online]. Available: <https://www.anthology.com>
- [4] M. Dougiamas and P. C. Taylor, “Moodle: Using learning communities to create an open source course management system,” in *Proc. World Conf. Educational Multimedia, Hypermedia and Telecommunications (ED-MEDIA)*, 2003, pp. 171–178.
- [5] H. Kaur and R. Singh, “Design and development of a campus activity management system for higher education institutions,” *Int. J. Adv. Comput. Sci. Appl.*, vol. 10, no. 4, pp. 213–221, 2019.
- [6] M. S. Rahman, A. Hossain, and F. Islam, “A mobile-first student activity tracking framework for resource-constrained university environments in Bangladesh,” in *Proc. Int. Conf. Comput. Inf. Technol. (ICCIIT)*, Dhaka, Bangladesh, 2021, pp. 1–6.
- [7] D. Patel and S. Mehta, “A MERN stack-based student club and event management system for university campuses,” *J. Emerging Technol. Innov. Res.*, vol. 9, no. 2, pp. 445–453, 2022.
- [8] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.
- [9] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST standard for role-based access control,” *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001.
- [10] OWASP Foundation, “OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks,” 2021. [Online]. Available: <https://owasp.org/Top10/>
- [11] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.
- [12] J. Brooke, “SUS: A quick and dirty usability scale,” in P. W. Jordan, B. Thomas, B. A. Weerdmeester, and A. L. McClelland, Eds., *Usability Evaluation in Industry*. London, UK: Taylor & Francis, 1996, pp. 189–194.
- [13] A. Bangor, P. Kortum, and J. Miller, “Determining what individual SUS scores mean: Adding an adjective rating scale,” *J. Usability Stud.*, vol. 4, no. 3, pp. 114–123, May 2009.